

File System

The project architecture is structured with SOLID principles in mind, promoting modularity and clear separation of concerns. This is reflected in the directory structure, aiming for high cohesion within modules and low coupling between them (SRP).

```
/ |— backend/ # Focus on business logic and data access | |— src/ | | |— api/ #  
Handles external requests, mapping to internal services (SRP for request handling) | | |  
| | |— v1/ # Versioned API for extensibility (OCP) | | | |— controllers/ # Process  
requests and delegate to services (SRP for request processing) | | | | |—  
employeeController.js | | | | |— workEntryController.js | | | | |—  
payPeriodController.js | | | | |— paymentController.js | | | | |— routes/ # Define  
API routes (SRP for routing) | | | | |— employeeRoutes.js | | | | |—  
workEntryRoutes.js | | | | |— payPeriodRoutes.js | | | | |— paymentRoutes.js |  
| | | | |— index.js # API version entry point (SRP) | | |— auth/ # Handles authentication  
and authorization concerns (SRP for security) | | | |— authService.js | | | |—  
authMiddleware.js | | | |— permissionManager.js | | |— config/ # Configuration  
handling (SRP for configuration) | | | |— database.js | | | |— config.js # Environment  
configurations | | | |— s3.js | | |— database/ # Database interaction layer (SRP for  
data persistence) | | | |— models/ # Database schema definitions (SRP for data  
structure) | | | | |— employeeModel.js | | | | |— workEntryModel.js | | | | |—  
workEntryImageModel.js | | | | |— payPeriodModel.js | | | | |—  
weeklyPaymentModel.js | | | | |— migrations/ # Database schema evolution (SRP for  
schema changes) | | | | |— seeders/ # Initial data population (SRP) | | |— services/ #  
Core business logic (SRP for business rules) - depend on abstractions (DIP) | | | |—  
interfaces/ # Abstractions for dependencies (DIP, ISP) | | | | |— IEmployeeRepository.js  
| | | | |— IWorkEntryRepository.js | | | | |— IPaymentCalculator.js | | | | |—  
IImageStorage.js | | | | |— implementations/ # Concrete implementations of interfaces  
(DIP) | | | | |— employeeRepository.js # Depends on database model | | | | |—  
workEntryRepository.js # Depends on database model | | | | |—  
paymentCalculatorService.js # Core calculation logic | | | | |— s3ImageStorage.js #  
Interacts with S3 | | | | |— utils/ # Helper utilities (SRP for utility functions) | | | | |—  
errorHandler.js | | | | |— logger.js | | | | |— asyncQueue.js # For async tasks like image  
processing (SRP for task handling) | | | | |— server.js # Backend application entry point |  
| | |— .env.example | | |— package.json | | |— README.md | | |— mobile-app/ #  
Frontend for field employees | | |— src/ | | | |— components/ # Reusable UI components  
(SRP for component responsibility, OCP for extensibility) | | | | |— navigation/ # React  
Navigation setup (SRP for navigation) | | | | |— screens/ # Screen-level components (SRP  
for screen logic) | | | | |— Auth/ | | | | |— LoginScreen.js | | | | |— Employee/ #  
Employee-related screens (SRP for employee UI) | | | | |— EmployeeListScreen.js | | |  
| | | | |— EmployeeDetailScreen.js | | | | |— EmployeeFormScreen.js | | | | |—  
WorkEntry/ # Work Entry-related screens (SRP for work entry UI) | | | | |—  
WorkEntryFormScreen.js | | | | |— WorkEntryListScreen.js | | | | |—  
WorkEntryDetailScreen.js | | | | |— ClockIn/ # Future Feature | | | | |—  
ClockInScreen.js | | | | |— SettingsScreen.js | | | | |— state/ # State management (SRP  
for state slices) | | | | |— auth/ | | | | |— employees/ | | | | |— workEntries/ | | |
```

utils/ # Frontend utilities | | | — api.js # API client, depends on abstract API interface (DIP - conceptually) | | | — storage.js # Local storage abstraction (DIP) | | | — syncService.js # Handles offline data sync (SRP for sync logic) | | — App.js # Mobile app entry point | — package.json | — README.md | — web-app/ # Frontend for employers (Admin/Manager) | — public/ | — src/ | | — components/ # Reusable UI components (shared where possible with mobile or web-specific, SRP, OCP) | | — navigation/ # React Router setup (SRP) | | — pages/ # Page-level components (SRP) | | — Auth/ | | | — LoginPage.js | | | — DashboardPage.js # Aggregates data from various services (SRP) | | | — Employee/ # Employee management pages (SRP) | | | | — EmployeeListPage.js | | | | — EmployeeDetailPage.js | | | | — EmployeeFormPage.js | | | — WorkEntry/ # Work Entry review pages (SRP) | | | | — WorkEntryListPage.js | | | | — WorkEntryDetailPage.js | | | — Payments/ # Payment management pages (SRP) | | | | — PayPeriodListPage.js | | | | — WeeklyPaymentListPage.js | | | | — WeeklyPaymentDetailPage.js | | | — Reports/ # Future Feature | | | | — ReportsPage.js | | | — Task/ # Future Feature | | | | — TaskManagementPage.js | | | — Materials/ # Future Feature | | | | — MaterialManagementPage.js | | — state/ # State management (SRP for state slices) | | | — auth/ | | | — employees/ | | | — workEntries/ | | | — payments/ | | | — dashboard/ | | — utils/ # Frontend utilities | | | — api.js # API client | | — App.js # Web app entry point | — package.json | — README.md | — docs/ # Documentation | — architecture.md | — database_schema.sql | — api_spec.md # Following API design principles (SRP, OCP, ISP)

Feature Specifications

Feature 1 — Gestión de Personal (CRUD de Empleados)

Feature goal

Enable administrators and managers to effectively manage the essential information of field/project workers (employees). This includes creating, viewing, editing, and logically deleting employee profiles, laying the groundwork for task assignment and payment calculation.

API relationships

This feature primarily interacts with the `/api/v1/employees` endpoints on the backend. It forms the foundational data used by the `Registro Diario de Trabajo` and `Cálculo de Pago Semanal` features. The API design will adhere to RESTful principles, promoting a clear separation of concerns (SRP for each resource endpoint).

Detailed feature requirements

- List active employees with search (by name/role) and filtering capabilities.
- View a complete employee profile, including basic data and pay rate type (hourly/salary).
- Create new employees with basic data and pay rate type.
- Edit existing employee information.
- Logically delete employees (soft delete).
- Validate input data for integrity (name, role, rates, etc.) on the backend.

- Implement authorization logic ensuring only 'admin' and 'manager' roles can modify employee data. This aligns with ISP by ensuring clients (roles) only have access to the operations they need, and SRP by isolating authorization logic.
- Provide mobile screens for listing, viewing details, and the employee form.
- Manage local and global state for employee data in the frontend.
- Implement pagination, search, and filtering in the employee list frontend.

****Detailed implementation guide****

1. ****System Architecture Overview****

* ****Diagram/Description:**** The architecture involves distinct layers: Frontend (Mobile/Web), Backend API, and Database. This separation supports ****SRP**** by assigning a single primary responsibility to each layer (UI, Business Logic/API, Data Persistence). The Backend is designed as a modular monolith (as discussed in the architectural considerations), where modules (like Employee, Work Entry, Payment) represent distinct areas of concern, further applying ****SRP**** within the backend. The use of Node.js/Express allows for a flexible structure that can be extended with new modules (aligning with ****OCP****).

* ****Stack Selection:**** React Native/React for frontend provides component-based architecture aiding ****SRP**** and reusability (****OCP****). Node.js/Express facilitates building a modular backend. PostgreSQL is a robust database. ORM/Query Builder and JWT are standard tools supporting the chosen architecture.

* ****Deployment Scheme:**** Stateless backend instances promote horizontal scaling, aligning with ****SRP**** (each instance handles requests without relying on local state).

* ****External Integration Points:**** None for this feature.

2. ****Database Schema Design****

* ****ERD:**** The schema is designed with tables (`employees`, `roles`) that represent single concepts, supporting ****SRP**** at the data modeling level. Foreign keys (`employees.role`) establish clear relationships.

```

```mermaid
erDiagram
 employees {
 uuid id PK
 varchar name
 varchar role FK "employees relates to roles"
 varchar pay_rate_type ENUM("hourly", "salaried")
 decimal pay_rate
 timestamp created_at
 timestamp updated_at
 timestamp deleted_at
 }
 roles {
 varchar name PK
 text description
 }
 employees ||--o{ work_entries : has
 employees ||--o{ weekly_payments : is_paid_in

```

roles ||--o{ employees : has\_role  
...

\* **Table Definitions:** Each table has a single, well-defined purpose (**SRP**). Columns within a table are highly related to the entity it represents (high cohesion).

\* **Indices and Foreign Keys:** Indices (``deleted_at``, ``role``, ``name``) are added to improve query performance, a technical detail, but the structure they support (efficient retrieval for specific criteria) enables the business logic's requirements. Foreign keys enforce relational integrity, ensuring data consistency related to the defined entities.

\* **Migrations/Versioning:** Using a migration tool allows schema changes to be applied incrementally and predictably. This supports **OCP** for the database schema itself – new changes are added as new migrations, minimizing modification of existing, applied migration scripts.

### 3. **Comprehensive API Design**

\* **REST Endpoints:** Endpoints are resource-oriented (``/employees``), with standard HTTP methods (GET, POST, PUT, DELETE) mapping to CRUD operations. This clear mapping supports **SRP**, dedicating each endpoint/method combination to a specific task. API versioning (``/v1``) ensures that future changes don't break existing clients, adhering to **OCP**.

\* **Auth and Permissions:** Authentication middleware and authorization logic are separate modules (**SRP**). Permission checks are implemented using a ``permissionManager`` service that the controllers depend on via abstraction (or simply by calling a dedicated function), illustrating **DIP** conceptually – controllers depend on the *concept* of permission checking, not a specific hardcoded implementation. The role-based permissions (Admin, Manager, Employee) mean that the API implicitly provides different interfaces or views of data based on the user's role, aligning with **ISP** where clients (roles) only "see" or can access the data/actions relevant to them.

\* **Error Handling & Status Codes:** Consistent error responses and standard HTTP codes provide a clear contract for API consumers, improving usability and making the system easier to understand and maintain. Error handling is centralized (**SRP** for error processing) and can be made pluggable (**OCP/DIP**).

\* **Rate Limiting and Caching:** These are technical details implemented as middleware or separate services (**SRP**), acting as concerns orthogonal to the core business logic.

### 4. **Frontend Architecture**

\* **Component Hierarchy:** The breakdown into Screens/Pages, Containers, and Presentational Components follows **SRP**, assigning distinct responsibilities (routing/layout, data fetching/logic, pure UI rendering). Reusable components further adhere to **SRP** (each component has a single UI responsibility) and **OCP** (they can be used in different contexts without modification).

\* **Reusable Component Library:** A dedicated library promotes **OCP** by allowing components to be extended or combined without altering their source code.

\* **State Management:** Using libraries for global state management (like Zustand or Redux Toolkit) promotes **SRP** by allowing the state logic for different domains (employees, auth) to be managed in separate "slices" or stores. Local state for form inputs adheres to **SRP** within the component.

\* **Routing:** A dedicated routing module/setup ensures **SRP** for navigation logic.

\* **Responsive Design:** Adapting UI for different screen sizes is a cross-cutting concern, but the underlying components should maintain their **SRP**.

## 5. **Detailed CRUD Operations**

\* **Create/Update:** Backend validation logic is separated from the core business logic (**SRP**). Authorization checks before performing the operation ensure that the principle of least privilege is enforced.

\* **Read:** Filtering and pagination logic is handled by dedicated functions or methods in the service/repository layer (**SRP**), preventing the controller from having too many responsibilities.

\* **Delete (Soft Delete):** Implementing soft delete (setting a `deleted\_at` timestamp) instead of hard deletion is a business rule requirement, but the mechanism (updating a field) is a specific database operation handled by the repository layer (**SRP** for data persistence). This doesn't directly relate to LSP in the traditional sense (subtypes), but ensures the "delete" operation behaves predictably and consistently.

## 6. **User Experience Flow**

\* **User Journey/Wireframes:** Designing clear user flows and interfaces, while not directly a SOLID principle, aligns with the spirit of **SRP** by aiming for UI elements and flows that have a single, clear purpose and are easy for the user to understand and use.

\* **State Transitions/Error Handling UI:** Visual feedback for loading, success, and error states helps the user understand the system's status. Centralizing UI error handling logic supports **SRP**.

## 7. **Security Considerations**

\* **Authentication/Authorization:** Authentication and authorization are handled by dedicated modules (**SRP**). The `permissionManager` service used for authorization embodies **DIP**, as the controllers rely on this service interface rather than a specific implementation detail of how permissions are checked. The role-based access control matrix reflects how different actors (roles) interact with the system, aligning with **ISP** by defining different sets of capabilities available to different clients.

\* **Protection CSRF, XSS, etc.:** These are cross-cutting security concerns implemented using middleware or specific libraries, separating these responsibilities from the core business logic (**SRP**).

## 8. **Testing Strategy**

\* **Unit, Integration, E2E Tests:** The modular architecture (enabled by **SRP**, **DIP**, **ISP**) significantly improves testability. Unit tests focus on individual components/functions (**SRP**). Integration tests verify interactions between modules, relying on well-defined interfaces. E2E tests cover the entire system flow. **DIP** is crucial for unit testing backend services, allowing mocks or stubs of dependencies (like repositories, storage services) to be injected.

\* **Performance Tests:** These are specific tests targeting non-functional requirements, handled separately (**SRP** for testing concerns).

## 9. **Data Management**

\* **Lifecycle Policies:** Data retention policies are business rules, but their implementation relies on data access logic handled by repositories (**SRP** for data management).

\* **Caching:** Caching mechanisms are implemented as separate components or layers (**SRP**) and interacted with through defined interfaces or patterns, potentially applying **DIP**.

\* **Real-time Requirements:** Addressed by specific technologies/patterns (e.g., WebSockets) in dedicated modules (**SRP**).

## 10. **Error Handling & Logging**

\* **Log Format/Classification:** Centralized error handling and structured logging facilitate monitoring and debugging. This promotes **SRP** by consolidating error processing logic. The logging system can be designed with interfaces (**DIP**) to allow switching logging implementations easily (**OCP**).

\* **Monitoring, Alerts & Recovery:** These are operational concerns separate from core application logic (**SRP**).

## ### Feature 2 — Registro Diario de Trabajo con Imágenes

### **Feature goal**

Allow field workers to record their daily activities, including hours worked, description, and upload images as visual evidence. Administrators and managers can record entries on behalf of others and review all work entries. This data is crucial for validating work and calculating payments.

### **API relationships**

This feature interacts with the `/api/v1/work-entries`` and `/api/v1/work-entries/:id/images`` endpoints. It depends on the `employees`` data for associating entries with workers and utilizes the Object Storage Service for image storage. The API design follows REST principles for clear resource management (**SRP**).

### **Detailed feature requirements**

- Provide a form to record date, description, hours worked, and optional location.
- Allow attaching one or multiple images to each entry.
- Provide image preview before upload and when viewing details.
- Display a history of work entries (filterable by employee and date for admin/manager).
- Allow editing existing entries and managing associated images (add/remove).
- Implement logical deletion (soft delete) of work entries and their associated images.
- Automatically capture geo-reference and timestamp on photos taken within the app (client-side).
- Validate input data for text fields and files (type, size) on the backend. Validation logic is separate (**SRP**).
- Implement authorization logic: employees only see/edit their own entries; admin/manager can see/edit/delete any entry. This applies **ISP** (different access levels for different roles) and **SRP** (authorization logic is distinct).
- Handle secure and efficient file uploads (images) to the Object Storage. This is a distinct technical concern (**SRP**).
- Store image URLs and metadata (geo, timestamp) in the database.
- Implement soft delete with cascading effect for work entries and images.
- Provide mobile components for the entry form, image picker, gallery, and viewer. Components have single responsibilities (**SRP**).

- Display photos clearly with geo-reference and timestamp overlay/metadata.
- Offline capability: Mobile app must allow creating entries and taking photos offline, syncing data upon connection. Offline sync is a separate module/service (\*\*SRP\*\*).

## **\*\*Detailed implementation guide\*\***

### **1. \*\*System Architecture Overview\*\***

\* **Diagram/Description:** The architecture extends Feature 1's with an Object Storage service and an asynchronous processing mechanism (message queue + worker). Storing images is a separate concern from storing entry text data (\*\*SRP\*\*). Asynchronous image processing prevents blocking the main request thread (\*\*SRP\*\*). The mobile app includes an offline layer, representing another distinct responsibility (\*\*SRP\*\*). Interaction with Object Storage is done via an abstraction (\*\*DIP\*\*) to allow changing storage providers in the future (\*\*OCP\*\*).

\* **Stack Selection:** Addition of Object Storage (S3 compatible) and a mechanism for asynchronous tasks (e.g., leveraging an existing queue/worker system or adding a new one like Redis Queue/BullMQ) aligns with \*\*SRP\*\* by separating time-consuming tasks.

\* **Deployment Scheme:** Separate worker processes are deployed to handle the image queue, ensuring the main API instances remain responsive. This modular deployment supports scaling specific concerns independently.

\* **External Integration Points:** Geolocation API (mobile OS), Object Storage Service (e.g., AWS S3).

### **2. \*\*Database Schema Design\*\***

\* **ERD:** New tables `work\_entries` and `work\_entry\_images` represent distinct entities (\*\*SRP\*\*). The one-to-many relationship between entries and images is clearly defined.

```

```mermaid
erDiagram
    employees {
        uuid id PK
        ...
    }
    work_entries {
        uuid id PK
        uuid employee_id FK "work_entries belongs to employee"
        date entry_date
        text description
        decimal hours_worked
        jsonb location # { latitude, longitude, accuracy }
        timestamp created_at
        timestamp updated_at
        timestamp deleted_at
    }
    work_entry_images {
        uuid id PK
        uuid work_entry_id FK "work_entry_images belongs to work_entry"
        varchar image_url
        varchar thumbnail_url # Optional, for optimization
    }

```

```

        jsonb geo_data # { latitude, longitude }
        timestamp photo_timestamp
        timestamp created_at
        timestamp updated_at
        timestamp deleted_at
    }
    employees ||--o{ work_entries : registers
    work_entries ||--o{ work_entry_images : includes
    ...

```

* **Table Definitions:** Each table is designed around a single concept (**SRP**). The `deleted\_at` timestamp supports soft delete, a state change managed consistently.

* **Indices and Foreign Keys:** Indices support efficient querying for work entries by employee and date range, important for the list views and payment calculation. The foreign key from `work\_entry\_images` to `work\_entries` defines the relationship. Soft deletion is handled by application logic rather than database cascade to maintain historical integrity and align with **SRP** (database is for persistence, application for business rules including soft delete relationships).

* **Migrations/Versioning:** Managed via migration tool (**OCP**).

3. **Comprehensive API Design**

* **REST Endpoints:** Clear endpoints for work entries and images follow **SRP**. Multipart requests for image uploads are handled by specific endpoints/logic (**SRP**). API versioning (**OCP**).

* **Auth and Permissions:** JWT authentication. Authorization logic in controllers/services enforces permissions based on roles, applying **ISP** (different capabilities per role) and relying on the separate auth module (**SRP**, **DIP**).

* **Error Handling & Status Codes:** Consistent responses for validation, auth, file upload errors. Centralized error handling (**SRP**).

* **Rate Limiting and Caching:** Apply rate limiting. Caching for frequently accessed lists.

4. **Frontend Architecture**

* **Component Hierarchy:** Specific components for image handling (picker, preview, gallery) have focused responsibilities (**SRP**). Leveraging reusable form and list components (**OCP**).

* **Reusable Component Library:** Extended with image-related components.

* **State Management:** Manage state for forms, image selection, and lists. Offline data state is a distinct concern managed separately (**SRP**).

* **Routing:** Standard navigation.

* **Responsive Design:** Mobile-first design for data entry, with web views optimized for review and filtering.

5. **Detailed CRUD Operations**

* **Create:** Frontend validation (basic type/size). Backend validation (mandatory fields, file type/size, employee authorization, unique constraint if applicable). Image upload initiated after validation, potentially via async process. This separates validation (**SRP**), core entry creation (**SRP**), and image handling (**SRP**).

* **Read:** Filtering by employee and date range. Pagination. Backend query logic in repository/service (**SRP**).

* **Update:** Updating text fields and managing images (add/remove) are distinct operations or parts of an operation. Soft deleting images when removed from an entry is a specific action handled by the service layer (**SRP**). Authorization checks based on role and edit window.

* **Delete (Soft Delete):** Soft deleting an entry involves updating its `deleted\_at`. Soft deleting associated images is a subsequent step handled by the application logic (**SRP** for the cascading soft delete rule), ensuring both are marked as deleted.

6. **User Experience Flow**

* **User Journey/Wireframes:** Clear flows for creating entries and reviewing them. Separate steps for adding images within the form. Displaying images and metadata distinctly in the detail view.

* **State Transitions/Error Handling UI:** Loading indicators for uploads and saves. Visual feedback for validation errors and sync status. Handling offline UI.

7. **Security Considerations**

* **Authentication/Authorization:** JWT. Role-based permissions enforced via backend logic (**ISP**, **SRP**, **DIP**).

* **Protection CSRF, XSS, etc.:** Sanitize text inputs. Strict validation on file uploads (type, size). Secure handling of image files and URLs. If geo-location is sensitive, consider access controls for that specific data point. Access control for image URLs (e.g., signed URLs) if required.

8. **Testing Strategy**

* **Unit Tests:** Test validation logic, image processing functions, repository methods for saving entries/images. Test offline storage logic.

* **Integration Tests:** Test API endpoints, including multipart uploads. Test interaction with Object Storage mock. Test the async processing queue integration. Test sync service logic.

* **E2E Tests:** Test creating entries with photos (including offline simulation), viewing them, and administrative review/modification.

* **Performance Tests:** Stress test upload endpoints and list endpoints. Test sync service performance with large volumes of offline data.

9. **Data Management**

* **Lifecycle Policies:** Soft-deleted entries/images are retained based on policy. Cleanup process managed separately (**SRP**).

* **Caching:** Cache work entry lists for faster viewing, especially in the web app filters. Client-side caching of images.

* **Real-time Requirements:** Not required for V1, but viewing active work status (future dashboard feature) might need real-time updates on new entries.

* **Offline Data (Mobile):** Implemented using local storage and a sync service (**SRP**). The sync service handles uploading data when online and resolving potential conflicts (implementing a clear strategy like "last write wins" for simplicity in V1). Backend must handle receiving data with client-generated IDs and manage potential duplicates during sync.

10. **Error Handling & Logging**

* **Log Format/Classification:** Structured logs for all events, including detailed info for uploads (file metadata). Specific error classification for file issues, sync conflicts, geo-location errors.

* **Monitoring, Alerts & Recovery:** Monitor API errors, storage service, async queue/worker status. Alert on upload failures, queue processing errors, sync issues. Document recovery for lost images or sync data. Monitor offline data accumulation.

Feature 3 — Cálculo de Pago Semanal

Feature goal

Automate the calculation of weekly payments for each employee based on registered work hours (for hourly employees) or a fixed salary (for salaried employees), considering valid work entries within a defined weekly pay period.

API relationships

This feature interacts with `/api/v1/pay-periods`` and `/api/v1/weekly-payments`` endpoints. It heavily depends on data from the ``employees`` and ``work_entries`` features. The calculation process is a core piece of business logic, separated into its own service (**SRP**).

Detailed feature requirements

- Define and manage weekly pay periods (start/end dates, status: draft, calculated, approved).
- Allow triggering the calculation process for a specific pay period (admin only). The trigger is a separate API endpoint (**SRP**).
- Sum valid work hours per employee for the period (for hourly). This is part of the calculation logic (**SRP** within calculation).
- Apply hourly rate or fixed weekly salary. Handled by the calculation logic.
- Store calculation results per employee and period. Data persistence is a separate concern handled by repositories (**SRP**, **DIP**).
- Visualize calculated payments by pay period (admin/manager). Retrieval logic is separate (**SRP**).
- Visualize the detail of a calculated payment.
- Allow manual adjustments to the calculated amount and adding notes (admin only). Adjustment logic is distinct (**SRP**).
- Mark states for pay periods (draft, calculated, approved). State transitions managed by specific endpoints/logic (**SRP**).
- Implement authorization: admin triggers/adjusts, admin/manager view all payments, employee views only their own. Applies **ISP** (different capabilities per role) and relies on auth module (**SRP**, **DIP**).
- Do not implement deletion for pay periods or payments to maintain history. This is a business rule.
- Provide web/mobile screens (primarily web for admin) for listing periods, listing payments by period, and viewing/adjusting payment details.
- Generate pay periods automatically (e.g., weekly) with an option for manual creation. Automatic generation is a scheduled task (**SRP**).
- Implement a configurable calculation engine for simple extras/discounts based on task type or fixed bonuses. The engine logic is separate (**SRP**), and its configuration should

allow extensions without modifying core calculation code (**OCP**) and potentially depend on abstractions for applying rules (**DIP**).

****Detailed implementation guide****

1. ****System Architecture Overview****

* ****Diagram/Description:**** Backend API with a dedicated `paymentCalculatorService` (**SRP**). This service interacts with `employeeRepository` and `workEntryRepository` via interfaces (**DIP**). Payment data is stored via `weeklyPaymentRepository` (also via interface - **DIP**). Pay period management is another related but distinct module/service (**SRP**). Automatic period generation is a scheduled task running independently (**SRP**). The configurable engine adds complexity but is encapsulated within the `paymentCalculatorService` or a related module (**SRP**).

* ****Stack Selection:**** Standard Node.js/Express, PostgreSQL. Scheduled task runner (e.g., node-cron, or OS-level cron) for auto-generation.

* ****Deployment Scheme:**** Scheduled tasks run on a designated backend instance or serverless function. Calculation process triggered via API call might run in the background (queue) for large datasets, but for V1 can be synchronous if performance is acceptable for typical periods. ****Decision:**** Start synchronous, refactor to async queue if needed based on performance.

* ****External Integration Points:**** None currently.

2. ****Database Schema Design****

* ****ERD:**** New tables `pay\_periods` and `weekly\_payments` represent the core entities for this feature (**SRP**). Relationships to `employees` are established.

```
```mermaid
erDiagram
 employees {
 uuid id PK
 ...
 }
 work_entries {
 uuid id PK
 ...
 }
 pay_periods {
 uuid id PK
 date start_date
 date end_date
 varchar status ENUM("draft", "calculated", "approved")
 timestamp generated_at
 timestamp calculated_at
 timestamp approved_at
 timestamp created_at
 timestamp updated_at
 }
 weekly_payments {
 uuid id PK
 }
```

```

 uuid pay_period_id FK "weekly_payments belongs to pay_period"
 uuid employee_id FK "weekly_payments belongs to employee"
 decimal calculated_amount
 decimal manual_adjustment # Can be positive or negative
 text adjustment_notes # Reason for adjustment
 decimal final_amount # calculated_amount + manual_adjustment
 timestamp created_at
 timestamp updated_at
 }
 employees ||--o{ weekly_payments : is_paid_in
 pay_periods ||--o{ weekly_payments : includes
 pay_periods { "weekly_payments are for this period" }
 ...

```

\* \*\*Table Definitions:\*\* Each table serves a specific purpose (\*\*SRP\*\*). `pay\_periods` tracks period status. `weekly\_payments` stores payment data, including adjustment details. Unique constraint on `(pay\_period\_id, employee\_id)` in `weekly\_payments` prevents data duplication related to the single responsibility of storing a payment for a specific employee and period.

\* \*\*Indices and Foreign Keys:\*\* Indices support querying pay periods by date/status and payments by period/employee. Foreign keys ensure relational integrity.

\* \*\*Migrations/Versioning:\*\* Managed via migration tool (\*\*OCP\*\*).

#### 4. \*\*Frontend Architecture\*\*

\* \*\*Component Hierarchy:\*\* Components for displaying pay period lists, payment tables, and a payment detail view with adjustment fields. These components are focused on their display/input responsibilities (\*\*SRP\*\*).

\* \*\*Reusable Component Library:\*\* Utilizes table, form, and date picker components.

\* \*\*State Management:\*\* State for lists, selected period/payment, and form data for adjustments.

\* \*\*Routing:\*\* Navigation between payment views.

\* \*\*Responsive Design:\*\* Primarily web-focused for administrative tasks.

#### 5. \*\*Detailed CRUD Operations\*\*

\* \*\*Create (Pay Period):\*\* Manual creation via API endpoint (\*\*SRP\*\*). Automatic creation via scheduled task (\*\*SRP\*\*). Backend validation on dates.

\* \*\*Read:\*\* Pay period lists (filtered by status) and payment lists (filtered by period/role) are fetched via dedicated API endpoints (\*\*SRP\*\*). Payment detail fetched by ID.

\* \*\*Update:\*\* Pay period status updates via specific API endpoints (\*\*SRP\*\*). Payment adjustments via specific API endpoint (\*\*SRP\*\*). Backend validation ensures state transitions are valid (e.g., cannot calculate approved period) and only admin can adjust/approve.

\* \*\*Delete:\*\* Not implemented, aligns with business requirement to maintain history.

#### 6. \*\*User Experience Flow\*\*

\* \*\*User Journey:\*\* Flows for admin/manager to manage periods and payments, and employee to view their payments.

\* \*\*Wireframes:\*\* Table views and detail views. Admin view includes extra buttons/fields for trigger, approve, adjust.

\* \*\*State Transitions:\*\* Loading states, visual indicators for period status. Feedback on calculation trigger and adjustment saves.

\* \*\*Error Handling UI:\*\* Inform user of invalid date ranges, unauthorized actions, or failed operations (e.g., calculation errors).

## 7. \*\*Security Considerations\*\*

\* \*\*Authentication/Authorization:\*\* JWT. Role-based permissions strictly enforced in backend API (\*\*ISP\*\*, \*\*SRP\*\*, \*\*DIP\*\*). The admin role has exclusive access to trigger calculation and make adjustments, enforcing the principle of least privilege for these sensitive operations.

\* \*\*Protection CSRF, XSS, etc.:\*\* Standard web security measures. Ensure sensitive payment data is handled securely.

## 8. \*\*Testing Strategy\*\*

\* \*\*Unit Tests:\*\* \*\*Crucial\*\* for the calculation logic. Test various scenarios: hourly employees with different hours, salaried employees, combinations, edge cases (0 hours, fractional hours), and applying simple extras/discounts using the configurable engine. Test pay period date logic.

\* \*\*Integration Tests:\*\* Test API endpoints for period/payment management and state transitions. Test the integration between the API trigger and the calculation service. Test authorization.

\* \*\*E2E Tests:\*\* Test the flow from work entry creation to viewing the calculated payment for an employee. Test admin triggering calculation and making adjustments.

\* \*\*Performance Tests:\*\* Test calculation time with a representative number of employees and work entries.

## 9. \*\*Data Management\*\*

\* \*\*Lifecycle Policies:\*\* Payment and pay period data are historical and retained indefinitely.

\* \*\*Caching:\*\* Cache pay period lists.

\* \*\*Real-time Requirements:\*\* None.

## 10. \*\*Error Handling & Logging\*\*

\* \*\*Log Format/Classification:\*\* Structured logs. Log calculation process start/end, duration, number of employees processed, and any errors during calculation (e.g., missing employee data, invalid work entries). Log manual adjustments (user, amount, notes).

\* \*\*Error Classification:\*\* Calculation errors, validation errors, authorization errors, state transition errors.

\* \*\*Monitoring, Alerts & Recovery:\*\* Monitor calculation process status and errors. Alert on calculation failures or prolonged processing times. Monitor scheduled task for auto period generation. Document recovery for calculation errors (manual re-calculation/correction).

## ### Feature 4 — Panel de Control del Empleador (Web)

### \*\*Feature goal\*\*

Provide employers (admins and managers) with a centralized web interface to oversee field operations, manage personnel and payroll, and access reports and key KPIs. This acts as the primary administrative hub.

## **\*\*API relationships\*\***

This feature consumes data from the `Gestión de Personal`, `Registro Diario de Trabajo con Imágenes`, and `Cálculo de Pago Semanal` features via their respective API endpoints. It will also interact with future features like Task and Material Management APIs. The web panel integrates these features, acting as a high-level client. It adheres to **\*\*ISP\*\*** by providing different views and capabilities based on the user's role (Admin vs. Manager).

## **\*\*Detailed feature requirements\*\***

- Visual dashboard summarizing daily/weekly status (e.g., active employees, work entry progress, pending approvals). The dashboard logic is a separate concern (**\*\*SRP\*\***).
- Access to full Personnel Management (CRUD - Feature 1). This is achieved by consuming Feature 1's API.
- Access to review and filter all Work Entries (Feature 2). Achieved by consuming Feature 2's API.
- Access to manage and calculate Weekly Payments (Feature 3). Achieved by consuming Feature 3's API.
- Visualization of Custom Professional Reports with key KPIs. Reporting is a separate concern (**\*\*SRP\*\***). The reporting module/engine should be designed to be extensible (**\*\*OCP\*\***) to add new report types. The logic for generating report data is separate (**\*\*SRP\*\***) and may depend on various services/repositories via interfaces (**\*\*DIP\*\***).
- Interface for advanced Task Management (assignment, tracking, forecasting) - **\*\*Future Feature Integration\*\***. Designed as separate modules/services (**\*\*SRP\*\***).
- Integrated flow for Materials Management and Request (catalog, requests, approvals) - **\*\*Future Feature Integration\*\***. Designed as separate modules/services (**\*\*SRP\*\***).
- Configurable "No-Code" interface for the Payment Calculation Engine (variables, rules) - **\*\*Future Feature (advanced part of F3)\*\***. This configuration UI is a separate frontend component (**\*\*SRP\*\***), interacting with a backend service responsible for managing calculation rules (**\*\*SRP\*\***). This backend service should allow adding new rule types (**\*\*OCP\*\***) and storing/retrieving rules (via repository - **\*\*DIP\*\***).
- Implement web authentication for the employer role. Relies on the shared auth module (**\*\*SRP\*\***).
- Develop UI/UX adapted for large screens for managing tables and dashboards.
- Integrate with the same backend endpoints used by the mobile app. This demonstrates how multiple clients can depend on the same backend interfaces, supporting **\*\*OCP\*\*** (backend is closed for modification by new client types) and **\*\*ISP\*\*** (clients use only the parts of the API they need).
- Develop frontend/backend modules for Task and Materials (Future). These will be separate modules/services (**\*\*SRP\*\***).
- Design/implement backend logic to calculate and generate custom KPIs. Separate service/module (**\*\*SRP\*\***).
- Develop frontend visualization for KPIs and report generation (exportable, e.g., PDF, CSV). Frontend components dedicated to visualization and reporting (**\*\*SRP\*\***).
- Design/implement "No-Code" interface and backend logic for payment engine configuration. Separate frontend/backend modules (**\*\*SRP\*\***), backend logic should be extensible (**\*\*OCP\*\***).

## **\*\*Detailed implementation guide\*\***

## 1. **\*\*System Architecture Overview\*\***

\* **\*\*Diagram/Description:\*\*** The Web App is a client consuming the Backend API, just like the Mobile App. The Backend provides APIs for all the features. New modules for Dashboard Summary, KPIs, Reports, Task Management (Future), Materials Management (Future), and Payment Engine Configuration (Future) will be added to the backend, each with a specific responsibility (**\*\*SRP\*\***). These modules will interact with core services (Employees, Work Entries, Payments) and repositories via interfaces (**\*\*DIP\*\***).

\* **\*\*Stack Selection:\*\*** Standard Web Frontend (React) and Node.js/Express Backend. Addition of charting libraries (frontend) and potentially a reporting engine (backend or frontend depending on complexity).

\* **\*\*Deployment Scheme:\*\*** Standard web application deployment. Backend modules for new features are part of the existing backend deployment initially (modular monolith).

\* **\*\*External Integration Points:\*\*** Potential reporting services.

## 2. **\*\*Database Schema Design\*\***

\* **\*\*ERD:\*\*** Relies on existing schemas and will add new tables for future features (Tasks, Materials, Payment Rules). Each new table will follow **\*\*SRP\*\***.

\* **\*\*Table Definitions:\*\*** New tables for tasks, materials, inventory, material requests, and payment calculation rules will be added.

\* **\*\*Indices and Foreign Keys:\*\*** Added for new tables to support efficient querying.

## 3. **\*\*Comprehensive API Design\*\***

\* **\*\*REST Endpoints:\*\*** Consumes existing APIs. New APIs will be introduced for Dashboard Summary, Reports, Tasks (Future), Materials (Future), and Payment Engine Configuration (Future). Each new API will follow RESTful principles and **\*\*SRP\*\***. Versioning (`/v1`, `/v2` if significant changes occur) supports **\*\*OCP\*\***.

\* **\*\*Auth and Permissions:\*\*** JWT authentication. Authorization logic in backend enforces access based on 'admin' and 'manager' roles, applying **\*\*ISP\*\*** and relying on the auth module (**\*\*SRP\*\***, **\*\*DIP\*\***). Data access within features is also subject to their specific permission rules (e.g., manager seeing only their assigned employees' data).

\* **\*\*Error Handling & Status Codes:\*\*** Consistent. Specific errors for dashboard/reporting failures.

\* **\*\*Rate Limiting and Caching:\*\*** Applied to new endpoints. Caching for dashboard data and KPI results.

## 4. **\*\*Frontend Architecture\*\***

\* **\*\*Component Hierarchy:\*\*** Dedicated components for dashboard widgets, charts, tables, report configuration forms, task management views, materials management views, and the payment engine configuration interface. Each component has a single, clear UI purpose (**\*\*SRP\*\***). Reusable components are key (**\*\*OCP\*\***).

\* **\*\*Reusable Component Library:\*\*** Extended with charting, data table, and potentially complex form/rule-building components.

\* **\*\*State Management:\*\*** Global state for dashboard data, filtered lists, report parameters, and future feature data. Each domain manages its own state (**\*\*SRP\*\***).

\* **\*\*Routing:\*\*** Comprehensive navigation for all sections of the employer panel.

\* **\*\*Responsive Design:\*\*** Optimized for large screens.

## 5. **\*\*Detailed CRUD Operations\*\***

\* CRUD for core features (Personnel, Work Entries, Payments) is done by calling existing APIs. New features will have their own CRUD operations defined via new APIs and implemented in dedicated services/repositories (**\*\*SRP\*\***).

## 6. **\*\*User Experience Flow\*\***

\* **\*\*User Journey:\*\*** Admin/Manager navigating through the various sections of the panel, viewing data, performing actions, generating reports.

\* **\*\*Wireframes:\*\*** Dashboard layout, complex data tables with filtering/sorting, forms, modals, report builders.

\* **\*\*State Transitions:\*\*** Loading states, visual feedback. Handling errors gracefully in the UI.

\* **\*\*Error Handling UI:\*\*** Display API errors, report generation errors, or data loading issues in context.

## 7. **\*\*Security Considerations\*\***

\* **\*\*Authentication/Authorization:\*\*** JWT. Access to the entire web panel is restricted to 'admin' and 'manager'. Permissions within each section (Personnel, Work Entries, Payments, Reports, etc.) are enforced according to the rules of the underlying features, leveraging the backend's authorization logic (**\*\*ISP\*\***, **\*\*SRP\*\***, **\*\*DIP\*\***).

\* **\*\*Protection CSRF, XSS, etc.:\*\*** Comprehensive web security measures. Sanitize all data rendered. Secure API communication.

## 8. **\*\*Testing Strategy\*\***

\* **\*\*Unit Tests:\*\*** Test frontend components, backend logic for dashboard calculations, KPI generation, and reporting.

\* **\*\*Integration Tests:\*\*** Test frontend integration with all consumed APIs. Test backend modules for new features and their interactions with core services/repositories.

\* **\*\*E2E Tests:\*\*** Test full user flows through the web panel, verifying interactions with all features and data display. Test different roles.

\* **\*\*Performance Tests:\*\*** Test dashboard loading time, list filtering performance with large datasets, and report generation time.

## 9. **\*\*Data Management\*\***

\* **\*\*Lifecycle Policies:\*\*** Data originates from core features, following their policies. Report data might have specific retention.

\* **\*\*Caching:\*\*** Implement caching for dashboard summaries, KPIs, and frequently accessed report data.

\* **\*\*Real-time Requirements:\*\*** Dashboard summary might benefit from near real-time updates (using WebSockets/polling), but not strictly required for V1.

## 10. **\*\*Error Handling & Logging\*\***

\* **\*\*Log Format/Classification:\*\*** Structured logs. Log user actions, errors during data fetching or actions within the panel, report generation events/errors, KPI calculation errors.

\* **\*\*Error Classification:\*\*** API errors displayed in UI, backend errors (dashboard, reports, KPIs), frontend errors.

\* **\*\*Monitoring, Alerts & Recovery:\*\*** Monitor web application health and performance. Monitor API endpoints for the web panel. Alert on errors in dashboard/reporting modules.



